THE AUSTRALIAN NATIONAL UNIVERSITY

Second Semester Final Examination – November 7th, 2022

COMP3320/6464/HONS
HIGH PERFORMANCE SCIENTIFIC COMPUTATION

- Time allowed: 3 hours and 30 minutes (including submission time to Wattle)
- Permitted materials: open-book exam
- Exam questions total 100 marks
- All students answer all questions
- Clarity and conciseness in answers will be highly valued
- Marks may be lost for supplying irrelevant information

1. Short Questions [20 marks]

a) Each of the following statements are either true or false. Which are they? Give a few sentences to explain each of your answers.

i. Between the underflow limit and overflow limit there exist groups of floating-point numbers that are distributed uniformly within the group

[2 marks]

ii. If two real numbers are exactly representable as floating-point numbers, then the result of their floating-point sum will always be an exactly representable floating-point number, provided that the result is not larger/smaller than the overflow/underflow limit.

[2 marks]

iii. All floating-point operations have rounding error.

[2 marks]

iv. If an application code can be run in parallel on an arbitrarily large number cores, the strong speedup S(N) obtained by running the code in parallel on N cores is always larger or equal to the strong speedup S(N-1) obtained by running the code in parallel on N-1 cores. Carefully justify your answer.

[4 marks]

v. The Pthread memory and execution modes guarantee that all variables stored on a thread's stack are thread private.

[2 marks]

vi. On parallel multi-core shared memory architectures, the false sharing of a cache line can be removed by using suitable atomic operations.

[2 marks]

vii. The increase of uniprocessor performance due to Instruction Level Parallelism has capped out mainly due to the end of Dennard scaling.

[2 marks]

b) In the following, `x`, `y`, and `z` are double precision floating point arrays, while `a`, `b`, and `c` are double precision floating point scalar variables.

i)
```
for (i=0; i<n; i++) a = a + b*x[i]*y[i]*z[i];
```
ii)
```
for (i=0; i<n; i++) y[i+1] = a*x[i]+b*y[i];
```

Would one of these loops significantly outperform the other on a contemporary x86 processor such as the ones on the Gadi system used in this course? Provide a detailed explanation for your answer assuming that the cache uses a write-allocate, write-back policy, and that no vectorization is implemented by the compiler. What if vectorization was enabled?

[4 marks]

2. Shared Memory Programming with OpenMP [20 marks]

The following C routine was found, by profiling, to dominate the computation time in a larger program. Your colleague has attempted to parallelize the routine using OpenMP. Assume that there is no pointer aliasing.

```
void four_body_interaction(double a[], double b[], double r[], int n){
  int i, j, k, l, ij = 0, kl = 0, ijkl = 0;
  double fij, fkl, fijkl = 0.0;
  #pragma omp parallel
  for(i = 0; i < n; ++i){
    #pragma omp for schedule(static)
    for(j = 0; j <= i; ++j){
      ++ij;
      fij = a[ij];
      kl = 0;
      fijkl = 0.0;
      #pragma omp simd
      for(k = 0; k < n; ++k){
        for(l = 0; l <= k; ++l){
          ++kl;
          ++ijkl;
          fkl = b[kl];
          fijkl += fij*fkl;
          r[ij] = fijkl;
        }
      }
    }
  }
}
```

a) Write general expressions for the values of the variable `ij`, `kl`, and `ijkl` as functions of the iterates `i`, `j`, `k`, and `l`.

[4 marks]

b) How does the calculation scale with the parameter `n`? Provide an analytic expression in terms of `n`.

[4 marks]

c) The code when compiled with OpenMP enabled and run in parallel on multiple threads and using vectorization gives incorrect results. Explain why this is the case by discussing each error.

[6 marks]

d) Rewrite the routine so that it runs correctly in parallel using OpenMP and with high parallel efficiency and performance. You are free to change the code as you like, adding additional local variables if required, although code correctness and functionality must remain the same. Explain briefly why you performed each optimization. Pseudocode is fine provided the intent is clear, there is no need to produce compileable C code.

[8 marks]

3. High Performance Computer Architecture [20 marks]

a) The following C code performs a vector scaling and sum in double precision, also known as a DAXPY.

```
void daxpy(int n, double a, const double x[], double y[])
{
   for (int i = 0; i < n; i++) {
      y[i] = a*x[i] + y[i];
   }
}
```

The code is run on a 3.2 GHz in-order processor with the following characteristics

- A group of up to five instructions can be issued per cycle, with (a) up to two integer instructions, (b) up to one load instruction, (c) up to one store instruction, (d) up to 2 different floating-point instructions between floating-point additions and floating-point multiplication, (e) up to one branch instruction.
- The latency from cache for a double-precision load is 2 cycles, with store operations to cache also completing in 2 cycles. The latency of a floating-point instruction is 2 cycles, and all integer instructions, including branching, will take at most 1 cycle.

(i) Calculate the minimum number of cycles required to complete one iteration of the loop in the case that all data is in cache. Give the cycle time including the full load/store latency.

[3 marks]

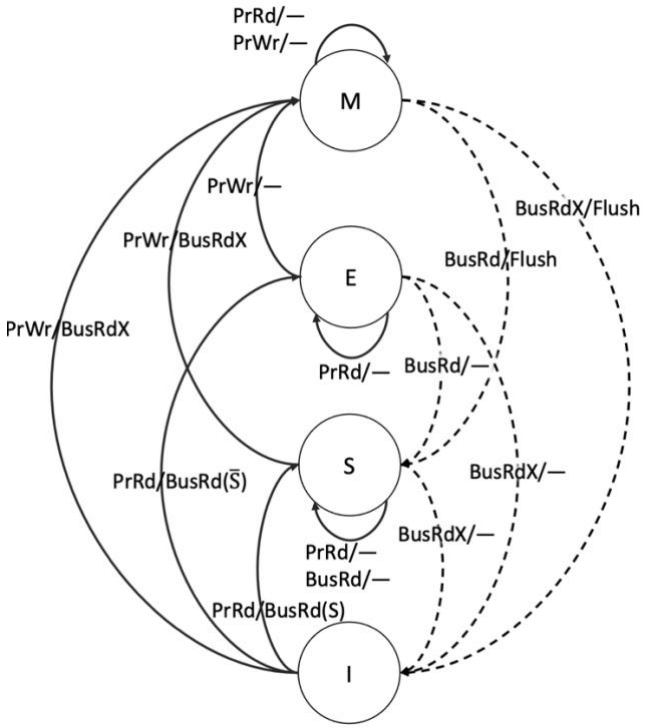
(ii) Your processor is now upgraded to perform fused floating-point multiply add (FFMA) operations with a latency of 3 cycles. What is the speedup achieved in executing the DAXPY code over the previous processor?

[3 marks]

(iii) What is the maximum FLOP rate achievable by the processor architecture at (ii), that is including FFMA, if loop unrolling is performed? Justify your answer and specify for which level of loop unrolling the maximum FLOP rate is achieved.

[6 marks]

b) Your machine implements (in hardware) the MESI cache coherence protocol shown in the following state diagram



The table below describes the behaviour of a program running in parallel using two processors P1 and P2 on your machine. C1 and C2 are cache lines associated with processors P1 and P2. The notation &Xi indicates the address of variable Xi (where i=1,2), while mem[&Xi] is the content of the memory cell (*i.e.* the value of Xi) at address &Xi. Note that the "bus transactions" are initiated as a consequence of the processor action and they correspond to the solid lines in the protocol (processor action/bus transaction), while the bus snooper transactions correspond to the dashed lines.

Following the MESI protocol, fill the entries in the table based on the listed processor actions with the assumption that X1 and X2 are stored contiguously and map to the same cache line.

processor action	P1 cache miss/hit, X value	P2 cache miss/hit, X value	Bus transaction (processor action initiated)	Bus snooper transaction	C1 state	C2 state	mem[&Xi]
No-Op	N/A	N/A	–	–	I	I	X1=0, X2=1
P1Rd X1	miss, 0	N/A	BusRd(S̄)	–			
P2Wr X1 = 1							
P1Wr X1 = X1+2							
P2Wr X1 = X1*2							
P1Rd X2							

[8 marks]

4. Application of High-Performance Computing [20 marks]

a) The following C code multiplies two $n \times n$ matrices **A** and **B** (stored in the 2-D arrays `A` and `B`) and stores the resulting matrix in array `C`. The matrix side dimension `n` is a large integer.

```
void matmul(int n, double **A, double **B, double **C){
    int i, j, k;
    for(k = 0; k < n; k++){
        for(j = 0; j < n; j++){
            for(i = 0; i < n; i++){
                C[i][j] += A[i][k]*B[k][j];
            }
        }
    }
}
```

Is the code above cache friendly? If not provide a new version of `matmul` that maximizes serial performance and discuss in detail your optimization choices. **[6 marks]**

b) Given a positive-definite symmetric matrix **A**, which is stored in the bidimensional array `a[1..n][1..n]`, the following routine constructs its Cholesky decomposition, $\mathbf{A} = \mathbf{L} \cdot \mathbf{L}^T$. On input, only the upper triangle of **A** needs to be given; it is not modified. The Cholesky factor **L** is returned in the lower triangle of **A**, except for its diagonal elements which are returned in `p[1..n]`.

```
void choldc(float** A, int n, float p[]) {
    int i, j, k;
    float sum;
    for (i = 0; i < n; i++) {
        for (j = i; j < n; j++) {
            k = i - 1;
            sum = A[i][j];
            while (k >= 0) {
                sum -= A[i][k] * A[j][k];
                k--;
            }
            if (i == j) {
                p[i] = sqrt(sum);
            } else {
                A[j][i] = sum / p[i];
            }
        }
    }
}
```

(i) Consider that you have a multi-core version of the CPU architecture. Write a parallel version of the code and discuss the potential performance improvements over its serial version. Motivate your parallelization strategy and implementation in detail. **[7 marks]**

(ii) Consider that the multi-core CPU architecture at point (ii) supports also both SSE2 and AVX-512 vector extensions. Discuss how you would improve the performance further (that is on top of thread parallelism) using SIMD instructions, carefully motivating your algorithm design choices. **[7 marks]**

5. Performance Modelling [20 marks]

You are given an application that code performs an elementwise sum, that is a reduction operation, of a one-dimensional array `v` and of a two-dimensional array `M`, each containing N and N^2 elements, respectively. Due to dependencies the reduction of `v` is not parallelizable, while the reduction of the `M` elements is parallelized over a number T of threads.

a) Assuming that t_a is the time taken by a single add, and that the total serial application time is determined only by timing of the sum operations, what are the serial and parallel fractions of runtime?

[6 marks]

b) Write expressions to model the strong and weak speedup of the code with respect to the number of threads T. For $N = 5$ and for $N = 100$, what is the maximum strong speedup S_s^{max} achievable by the code?

[6 marks]

c) A detailed performance analysis of the code reveals that, in addition to the total time taken by the elementwise sums, there is a runtime overhead associated with the parallel reduction of the elements of `M` that scales as $C(N, T, t_a) = 0.05\, N^2 log(T) t_a$. Write an updated expression for the strong speedup that accounts for the parallel reduction overhead and motivate your answer.

[6 marks]

d) Outline a strategy to compute the number of threads T that yields the maximum strong speedup obtained at point c).

[2 marks]